

## PX00-BASELINE-RECONCILIATION — Replace the Old Prompt Pack With Current Repo Reality

### CONTEXT

Project: Syrian Private University website ([spu.edu.sy](http://spu.edu.sy))

Stack: Laravel 12, Filament v3, MySQL 8, Redis

Architecture rule: Service Layer is the only place for business logic

The previous P01-P12 prompt pack is no longer an accurate execution plan for this repository.

This repo has already moved significantly beyond the original assumptions:

- interfaces, DTOs, middleware, auth scaffolding, schema, and many models already exist
- legacy import infrastructure exists and has already been heavily audited
- import/reporting commands already exist in [routes/console.php](#)
- public runtime is still mostly stubbed in [routes/web.php](#)
- Filament is already mounted under [/admin](#)
- launch-critical continuity is still missing:
  - no redirect continuity layer
  - no sitemap
  - no [robots.txt](#)
  - no public canonical/hreflang rendering
  - no file continuity layer
- placeholder service bindings still exist in [AppServiceProvider](#)
- [public/storage](#) is not linked in the current environment

This phase is not for implementing new public features yet.

This phase is for creating a clean repo-reality baseline so all future work is grounded in what already exists, what is partial, what is wrong, and what the actual critical path now is.

### TASK

Audit the current repository and replace the old prompt pack as the execution source of truth.

You must do all of the following:

1. Audit the old prompt pack against the current repo

Compare the historical work plan assumptions against actual repo reality.

At minimum, determine for each old prompt area whether it is:

- done
- partially done
- missing
- obsolete
- dangerous to rerun as originally written

Map the old work areas against actual existing code such as:

- contracts
- DTOs
- middleware
- auth/RBAC
- schema
- models
- core services
- homepage foundation
- public routing/runtime
- admin/Filament
- SEO
- caching
- tests
- legacy import tooling
- migration continuity

## 2. Produce a repo-reality baseline

Create a clear baseline summary that documents:

- what already exists and should be preserved
- what exists but needs patching
- what is still placeholder
- what has not been implemented yet
- what the real blockers now are
- what the current project execution phases should be

This baseline must be concrete and engineering-usable, not vague.

## 3. Replace the old execution plan

Create a replacement roadmap for continuing work in phases.

The new roadmap must be based on current repo state, not old greenfield assumptions.

It must explicitly separate:

- already-built foundation work
- incomplete runtime work
- homepage CMS completion
- navigation/settings completion
- SEO output + continuity layer
- admin/Filament completion
- migration backfill/tooling
- hardening/tests/launch readiness

## 4. Record critical project decisions that must be made

Explicitly identify and document the major decisions required before safe implementation continues, including at minimum:

- `/admin` strategy
- page URL strategy
- slug strategy
- locale strategy
- non-AR/EN legacy content strategy
- archive vs redirect vs 410 strategy
- rollback threshold / cutover threshold

For each decision, state:

- why it matters
- what code/schema/runtime it affects
- whether the repo currently implies a default
- whether the decision is still unresolved

## 5. Identify what must NOT be regenerated

Call out foundation areas that already exist and should be patched instead of rebuilt.

At minimum consider:

- contracts
- DTOs
- middleware
- auth scaffolding
- schema
- models
- import tooling
- import audit/export commands

## 6. Identify the actual critical path now

Based on the current repository, identify what is truly blocking migration readiness.

Be explicit and evidence-driven.

You should expect the critical path to center around things like:

- replacing stub public runtime
- real homepage/page rendering
- SEO output
- redirect continuity
- sitemap/robots
- file continuity
- admin path conflict
- launch validation and rollback prep

But do not assume these blindly; verify them from the repo.

## REQUIRED REPO INPUTS

You must derive this phase from actual current repository work, including at minimum:

- `routes/web.php`
- `routes/console.php`
- `app/Providers/AppServiceProvider.php`
- `app/Contracts/*`
- `app/DTOs/*`
- `app/Http/Middleware/*`
- `app/Models/*`
- `database/migrations/*`
- `database/seeder/*`
- existing import/audit/export commands
- current public/admin route reality
- current placeholder vs real service state
- current docs under `Docs/WEBSITE_MIGRATION_GUIDE`
- prior SEO migration audit conclusions already available in the repo/workspace context

## OUTPUT FORMAT

Return the result using this exact structure:

### 1. Current Repo Baseline

- what already exists
- what is partial
- what is missing
- what is obsolete/stale in the old pack

### 2. Old Prompt Pack Status Matrix

A concrete matrix for the old P01-P12 areas:

- prompt area
- status: done / partial / missing / obsolete / dangerous to rerun
- evidence
- recommended treatment

### 3. Critical Decisions

For each unresolved decision:

- decision
- why it matters
- affected files/tables/runtime behavior
- recommended default if no decision is made yet

#### 4. What Must Not Be Rebuilt

List the existing foundation pieces that should be patched, not regenerated.

#### 5. New Phase Roadmap

Provide the replacement phased plan in order.

#### 6. Real Critical Path

State the actual migration-readiness blockers in priority order.

#### 7. Recommended Immediate Next Step

Give the best next phase to execute after this reconciliation completes.

#### DONE WHEN

- The old prompt pack has been effectively replaced as the current source of truth.
- There is a clear done/partial/missing/obsolete view of the old P01-P12 areas.
- The critical unresolved decisions are explicit.
- The actual critical path is identified based on repo reality.
- There is a replacement phased roadmap that future prompts can follow safely.
- The result is actionable enough to drive the next implementation phase without relying on the old pack.

#### CONSTRAINTS

- Do not regenerate interfaces, schema, or models blindly.
- Do not assume early phases are greenfield.
- Prefer actual repository evidence over historical prompt assumptions.
- Do not implement new code in this phase unless absolutely necessary to support the baseline itself.
- Keep business-logic placement rules intact:
  - business logic in services
  - thin controllers
  - passive models

## PX01-FOUNDATION-NORMALIZATION — Patch Existing Foundation to Match Current Repo

Reality

DEPENDS ON

PX00-BASELINE-RECONCILIATION

CONTEXT

Project: Syrian Private University website ([spu.edu.sy](http://spu.edu.sy))

Stack: Laravel 12, Filament v3, MySQL 8, Redis

Architecture rule: Service Layer is the only place for business logic

This repository is not at a greenfield foundation stage anymore.

A significant part of the early foundation already exists and must now be normalized, patched, and aligned instead of being regenerated from scratch.

Current repo reality that must be treated as true:

- contracts already exist under [app/Contracts](#)
- DTOs already exist under [app/DTOs](#)
- middleware already exists under [app/Http/Middleware](#)
- auth/RBAC scaffolding already exists
- schema and migrations already exist and are applied
- many models already exist
- placeholder service bindings still exist in [AppServiceProvider](#)
- public runtime is still mostly stubbed and will be addressed later
- Filament is already mounted under [/admin](#)
- legacy import foundation and audit/export tooling already exist
- the old prompt pack assumed earlier phases were incomplete, but that is no longer true

This phase is for patching and normalizing the existing foundation so the next implementation phases can build on stable and intentional base behavior.

This phase is not for:

- implementing the full public runtime
- implementing redirects/sitemaps/robots
- implementing full admin UX
- rebuilding the schema from scratch

GOAL

Bring the current repository foundation into a clean, internally consistent, phase-ready state.

That means:

- contracts reflect actual intended scope
- DTOs reflect actual service boundaries
- bindings are intentional
- middleware/auth/model foundation is coherent
- seed policy is explicit
- stale/wide-scope assumptions are removed
- no one needs to "rerun old P01-P05" blindly anymore

## TASK

Patch and normalize the existing foundation.

You must do all of the following.

### 1. Review and patch existing contracts

Inspect all current contracts in [app/Contracts/](#).

Goals:

- remove stale or over-wide assumptions from earlier prompt generations
- keep only contracts that make sense for the current homepage/admin foundation and near-term migration work
- ensure all public service contracts use typed PHP 8.2 signatures
- ensure public service contracts do not return raw Eloquent models
- ensure DTO/scalar/collection return expectations are clear

At minimum evaluate and normalize contracts around:

- auth
- audit
- cache
- media
- menu
- navigation
- settings
- page handling
- homepage section/publishing
- preview
- SEO
- slug generation

If a contract is clearly obsolete or too wide for the current repo direction, patch or de-scope it instead of leaving it misleading.

### 2. Review and patch existing DTOs

Inspect [app/DTOs/](#).

Goals:

- ensure DTOs match current service boundaries and the actual next phases
- remove DTO drift caused by earlier wide-scope assumptions
- ensure naming, fields, imports, and readonly usage are consistent
- ensure DTOs support the actual runtime/admin/navigation/SEO/homepage needs

Do not invent broad new DTO layers unless the current service boundary actually requires them.

3. Normalize service bindings in `AppServiceProvider`

Inspect and patch `app/Providers/AppServiceProvider.php`.

Goals:

- explicitly review all interface-to-implementation bindings
- determine which bindings are still placeholders
- keep placeholders only where they are still intentionally temporary
- ensure bindings match the intended next phases
- ensure no interface resolves ambiguously or to stale placeholders by accident

You should not replace placeholders with full real implementations unless that is strictly required for foundation normalization.

This phase is about `binding hygiene`, not full service completion.

4. Normalize middleware and route-foundation expectations

Inspect:

- `app/Http/Middleware/*`
- `bootstrap/app.php`
- `routes/web.php`

Goals:

- confirm middleware aliases and usage are coherent
- patch naming/alias inconsistencies if present
- ensure locale/admin/cache middleware expectations match the actual current architecture
- ensure no stale assumptions remain from older prompt work
- ensure middleware is prepared for later phases without overreaching now

This phase should not replace the stub public runtime yet, but it should make sure the foundation around it is clean.

5. Normalize auth/RBAC foundation

Inspect existing auth-related code, including:

- auth service/contracts
- login request/controller if present
- gates/policies/providers
- user/role model behavior
- auth-related migrations and seeders

Goals:

- ensure role logic matches current intended scope
- ensure account lock fields and failed-attempt logic are coherent
- identify and patch drift such as duplicated fields or inconsistent naming where safe
- ensure the foundation is stable for the later admin/public work

Do not build new auth features outside the foundation scope.

6. Review and patch model helpers/scopes only as foundation cleanup

Inspect existing models under [app/Models](#).

Goals:

- keep models passive
- ensure relationships/scopes/helpers support actual current and next-phase needs
- remove or patch stale helpers that embed business assumptions
- ensure casts/fillable/soft deletes are aligned with the real schema
- ensure model methods do not compete with planned service-layer behavior

This is not a model redesign phase.

Patch only what is needed to normalize the foundation.

7. Normalize seeding policy

Inspect:

- [database/seeders/DatabaseSeeder.php](#)
- role/user seeders
- homepage/page/navigation/settings seeders
- any seeders that create placeholder/editor-facing content

Goals:

- explicitly separate the conceptual roles of:
  - local/dev scaffolding
  - migration/import support
  - production-safe seeding
- patch seeders if current behavior is misleading or unsafe
- ensure repeated local rebuilds remain reasonable
- ensure production-like environments are not implicitly seeded with placeholder launch content unless intentional

This phase should make the seeding story clear and safe.

#### 8. Remove stale full-site assumptions from the foundation

Review current foundation code for assumptions that belong to old full-site scope rather than the current phased build.

Examples:

- contracts implying full public modules too early
- DTOs implying modules not yet in scope
- bindings or helpers tied to broad legacy scope without current use
- seed defaults pretending to be final public IA/SEO behavior

Patch these where necessary so the foundation reflects the current real phased plan.

#### REQUIRED INPUTS

You must derive this phase from the actual current repository, including at minimum:

- `app/Contracts/*`
- `app/DTOs/*`
- `app/Providers/AppServiceProvider.php`
- `app/Models/*`
- `app/Http/Middleware/*`
- `bootstrap/app.php`
- `routes/web.php`
- auth-related controllers/requests/services/policies/gates
- `database/migrations/*`
- `database/seeder/*`

You must base your work on current repo reality, not the old greenfield P01-P05 assumptions.

#### REQUIRED OUTPUT OF THE WORK

By the end of this phase, the foundation should have these properties:

##### Contracts

- intentional, current-scope contracts only
- typed signatures
- no raw Eloquent model return types in public service contracts

##### DTOs

- internally consistent
- aligned to actual service boundaries
- no stale wide-scope leftovers

## Bindings

- interface resolution is clean and intentional
- placeholder vs real bindings are deliberate

## Middleware/auth/model foundation

- coherent naming and behavior
- no obvious drift/conflicting assumptions
- stable base for later runtime/admin work

## Seed policy

- clearly suitable for phased development
- local/dev scaffolding is distinct from launch-safe behavior

## VERIFICATION REQUIREMENTS

At the end of the phase, verify at minimum:

- `php artisan list` runs without container errors
- `php artisan route:list` runs cleanly
- all key interfaces resolve from the container
- no public contract returns raw Eloquent models
- foundation bindings are intentional and consistent
- no stale early-phase assumptions remain in core foundation code

## DONE WHEN

- Existing contracts have been patched to match the actual repo direction.
- Existing DTOs have been normalized to current service boundaries.
- `AppServiceProvider` bindings are clean and intentional.
- Middleware/auth/model foundations no longer reflect stale greenfield assumptions.
- Seeding policy is explicit enough to support future phases safely.
- The repository is ready for the next phase without rerunning old P01-P05 logic from scratch.

## CONSTRAINTS

- Patch existing work; do not regenerate the entire foundation.
- Do not rebuild schema from scratch.
- Do not implement the full public runtime yet.
- Do not implement redirect/sitemap/robots continuity yet.
- Do not move business logic into controllers or models.
- Do not expand this phase into full public module delivery.
- Prefer small, corrective, architecture-safe changes over broad rewrites.

## ARCHITECTURE RULES

- Service Layer is the only place for business logic.
- Controllers must stay thin.
- Models may contain only:
  - relationships
  - scopes
  - casts
  - simple accessors/mutators
  - tiny passive helpers
- Interfaces must not return raw models publicly.
- DTOs should be used where structured service output is needed.
- Prefer patching over replacing.

#### IMPORTANT IMPLEMENTATION NOTE

This phase exists because the old P01-P05 plan is no longer safe to execute literally.

Treat the current repo as the base truth.

Do not behave as if contracts, middleware, schema, models, and auth foundation are missing. They already exist and must be normalized deliberately.

## PX02-PUBLIC-RUNTIME — Replace Stub Public Runtime With Real Homepage and Landing-Page

Rendering

DEPENDS ON

PX01-FOUNDATION-NORMALIZATION

CONTEXT

Project: Syrian Private University website ([spu.edu.sy](http://spu.edu.sy))

Stack: Laravel 12, Filament v3, MySQL 8, Redis

Architecture rule: Service Layer is the only place for business logic

The repository already contains a meaningful content and CMS foundation, including:

- [pages](#)
- [page\\_translations](#)
- [page\\_seo\\_meta](#)
- [homepage\\_sections](#)
- [homepage\\_section\\_translations](#)
- [homepage\\_drafts](#)
- [preview\\_tokens](#)
- [menu\\_items](#)
- [settings](#)
- [media\\_assets](#)

However, the public runtime is still mostly stubbed.

Current repo reality that must be treated as true:

- [routes/web.php](#) still uses temporary/stub public closures
- [/](#) redirects to [/ar](#)
- [/{locale}](#) is not yet a real homepage rendering pipeline
- [/{locale}/preview](#) is not yet a real preview rendering flow
- public page-by-slug rendering is not yet complete
- seeded/imported content already exists and should now start being used
- Filament/admin already exists separately and is not the target of this phase
- SEO continuity work like redirects/sitemap/robots will come later

This phase is for replacing the stub public runtime with real service-backed homepage and landing-page rendering.

It is not for:

- redirect continuity
- sitemap
- [robots.txt](#)
- file continuity
- final Filament/admin UX

- full public modules like News, Events, Research repository, Admissions, Facilities, Contact CRM

## GOAL

Implement a real public runtime for:

- homepage rendering
- landing-page shell rendering
- page resolution by locale + slug
- breadcrumbs
- language-switch context preservation
- preview hydration without public leakage

The result should stop relying on stubbed locale-root responses and instead use the actual content foundation already in the database.

## TASK

Replace the stub public runtime with real controllers, routes, services, and views.

You must do all of the following.

1. Review current public route/runtime shape

Inspect the existing public runtime and replace the stub path cleanly.

At minimum inspect:

- [routes/web.php](#)
- locale middleware behavior
- existing public controllers if any
- existing placeholder service bindings
- page/homepage/navigation/settings/preview contracts and DTOs
- homepage/page-related models
- existing views and public layout structure

You must base this phase on current repo reality, not assume a greenfield public runtime.

2. Implement real homepage rendering

Create or complete a real homepage pipeline.

At minimum this should:

- resolve the locale from the current route context
- fetch the public homepage payload through the service layer
- fetch navigation payload
- fetch utility/footer/settings payload

- fetch SEO payload
- pass structured data to a real homepage view

The homepage must render using published and enabled data only.

Homepage routes must remain locale-prefixed:

- [/ar](#)
- [/en](#)

The homepage should no longer depend on inline closure HTML.

### 3. Implement real landing-page shell rendering

Create or complete real public page rendering by slug + locale.

At minimum:

- implement page lookup by locale + slug
- use the service layer to hydrate a public page DTO/payload
- build breadcrumb payload
- include locale-aware SEO payload
- include navigation/footer shell payload
- return a real landing-page shell view

Public page rules:

- missing page => 404
- disabled page => 404
- unpublished page => 404
- scheduled page before publish time => 404
- draft page => 404

### 4. Implement or complete the public controllers

Create or patch the public controllers needed for this phase.

At minimum:

- [HomeController](#)
- [PageController](#)

Controllers must:

- inject interfaces only
- not import raw models directly
- contain orchestration only
- not contain business logic
- return structured view payloads only

## 5. Implement or complete the required real services

Patch or complete the existing service implementations needed to support the public runtime.

At minimum this phase should complete enough real behavior in:

- `PageService`
- `NavigationService`
- `SettingsService`
- `SeoMetadataService`
- `PreviewService`
- homepage-related public retrieval service(s)

These services must:

- return DTOs or structured payloads
- not return raw Eloquent models publicly
- build the payloads needed by controllers/views
- support public read concerns only as needed for this phase

This phase is about making the public runtime work, not finishing all admin-side editing flows.

## 6. Resolve page URL behavior using repo reality

This phase must explicitly implement a consistent public URL model based on the current schema and repo direction.

You must inspect and decide how current runtime should work with the existing schema, especially:

- global `pages.slug`
- locale-prefixed URLs
- translation records in `page_translations`

At minimum, implement a consistent strategy for:

- homepage route behavior
- top-level landing page resolution
- child page resolution if hierarchy is already supported
- language-switch URL generation preserving current page context where equivalent content exists

Do not invent a second URL model without strong repo evidence.

## 7. Define and implement page content precedence in practice

The current schema supports multiple content-bearing structures:

- `pages.content_json`

- translation payload fields
- translation body/excerpt-like content if present

You must inspect the actual current models/schema/contracts and implement a clear runtime precedence for public page hydration.

This precedence must be:

- intentional
- documented in code comments where needed
- stable enough for later phases

The goal is to avoid ambiguous rendering behavior.

8. Implement preview hydration without public leakage

Patch or complete the preview flow so the public runtime can support preview-safe hydration.

Requirements:

- preview must remain tokenized
- preview must not expose draft content publicly
- preview must be locale-aware
- preview must support later device-mode use cleanly
- preview routes must remain distinct from normal public rendering rules

This phase does not need the full final preview UX, but it must establish the correct runtime foundation.

9. Add real public views/layout structure

Create or patch the public rendering layer so the real runtime can return actual views instead of stub responses.

Views should receive structured data only, such as:

- `navigation`
- `utilityNavigation`
- `footerNavigation` or `footerPayload`
- `seo`
- `locale`
- `direction`
- `homepage`
- `page`
- `breadcrumbs`
- `languageSwitch`

Do not push business logic into Blade templates.

10. Keep this phase limited to homepage + landing-page shell

This phase is only for:

- homepage
- generic landing-page shell
- shared public shell concerns needed by those pages

Do not expand into:

- full News module
- full Research repository
- full Events calendar
- full Facilities module
- full Admissions flow
- full Contact/CRM workflow

Card payloads or placeholder sections are acceptable where the homepage shell already depends on them, but do not convert this into full module delivery.

## REQUIRED INPUTS

You must derive this phase from the actual current repository, including at minimum:

- `routes/web.php`
- `app/Providers/AppServiceProvider.php`
- `app/Contracts/*`
- `app/DTOs/*`
- `app/Services/*`
- `app/Models/Page.php`
- `app/Models/PageTranslation.php`
- `app/Models/PageSeoMeta.php`
- `app/Models/HomepageSection.php`
- `app/Models/HomepageDraft.php`
- `app/Models/PreviewToken.php`
- `app/Models/MenuItem.php`
- `app/Models/Setting.php`
- `resources/views/*`
- homepage/page-related seed data already present
- locale middleware already present

You must build from the work already done in this repo.

## REQUIRED IMPLEMENTATION OUTCOMES

By the end of this phase, the public runtime should have these properties:

Homepage

- `/ar` renders from real data

- [/en](#) renders from real data
- homepage payload comes from real service-backed retrieval
- homepage uses published/enabled content only

## Landing pages

- landing pages resolve by locale + slug
- real page payload is built through services
- non-public page states are blocked
- breadcrumb payload is available
- language-switch context is preserved when possible

## Controllers

- thin, service-driven, no raw model orchestration
- no embedded business logic

## Services

- enough real logic exists to support public runtime
- DTO/structured output only
- no raw model returns publicly

## Views

- receive structured public payloads
- no business logic
- locale and direction are available in view context

## VERIFICATION REQUIREMENTS

At the end of the phase, verify at minimum:

- [/ar](#) renders successfully from real service/data flow
- [/en](#) renders successfully from real service/data flow
- at least one top-level landing page renders successfully from real page data
- missing page slug returns 404
- draft page is not publicly accessible
- scheduled page before publish time is not publicly accessible
- disabled/unpublished page is not publicly accessible
- breadcrumb payload exists on non-home landing pages
- language switch preserves page context where equivalent content exists
- public controllers do not import raw models directly
- stub closure rendering for the main public runtime is removed or no longer used for homepage/page shell routes

## DONE WHEN

- The stub public runtime has been replaced with real homepage and landing-page rendering.
- Public routes are backed by real controllers and services.
- Homepage and landing pages render from actual stored data.
- Public visibility rules are correctly enforced.
- Breadcrumb and locale-switch payloads are in place.
- The repository is ready for the next phase of homepage CMS completion on top of a real public runtime.

## CONSTRAINTS

- Do not implement redirect continuity in this phase.
- Do not implement sitemap in this phase.
- Do not implement `robots.txt` in this phase.
- Do not build full public feature modules outside the homepage/admin foundation.
- Do not return raw Eloquent models from public service methods.
- Do not put business logic in controllers or views.
- Do not rebuild existing schema from scratch.
- Patch and extend the existing repository foundation.

## ARCHITECTURE RULES

- Service Layer is the only place for business logic.
- Controllers must remain thin.
- Models must remain passive.
- Public services must return DTOs or structured payloads.
- Use the existing schema and repository work as the source of truth.
- Prefer minimal correct changes over broad rewrites.

## IMPORTANT IMPLEMENTATION NOTE

This phase assumes the foundation has already been normalized and that the current repo already contains meaningful content structures.

Do not behave as if homepage/page runtime is starting from zero.

You are patching a partially built system into a real public runtime.

## X03-HOMEPAGE-CMS — Finish Homepage CMS, Draft, Publish, Preview on Top of the Real

Runtime

DEPENDS ON

PX02-PUBLIC-RUNTIME

CONTEXT

Project: Syrian Private University website ([spu.edu.sy](http://spu.edu.sy))

Stack: Laravel 12, Filament v3, MySQL 8, Redis

Architecture rule: Service Layer is the only place for business logic

The repository already contains meaningful homepage foundation pieces, including:

- [homepage\\_sections](#)
- [homepage\\_section\\_translations](#)
- [homepage\\_drafts](#)
- [preview\\_tokens](#)

There is also already seeded homepage structure and section data scaffolding in the repo.

This phase is **not** greenfield homepage creation.

It is for completing and normalizing the homepage CMS workflow on top of the real public runtime implemented in PX02.

The homepage is a fixed 10-section CMS page and must use these exact keys only:

- [hero](#)
- [hero\\_stats](#)
- [academic\\_faculties](#)
- [achievements\\_highlights](#)
- [university\\_news](#)
- [research\\_studies](#)
- [events\\_activities](#)
- [medical\\_facilities\\_services](#)
- [bottom\\_stats](#)
- [footer](#)

Old legacy homepage block assumptions must not be reintroduced.

This phase must support:

- AR and EN independent homepage content
- structured payloads per section
- section validation
- draft save
- publish now
- unpublish

- schedule publish
- preview token workflow
- cache invalidation
- audit logging
- public hydration from published/enabled homepage data only

This phase is not for:

- full News module
- full Events module
- full Research repository
- full Facilities module
- full Admissions public module
- final Filament UX completion
- redirects/sitemap/robots continuity

## GOAL

Complete the homepage CMS so that the fixed homepage structure can be safely:

- edited
- validated
- saved as draft
- previewed
- published
- scheduled
- unpublished

And so that the public homepage runtime consumes that content safely and correctly.

## TASK

Implement or complete the homepage CMS workflow on top of the current repository foundation.

You must do all of the following.

1. Review the current homepage foundation already in the repo

Inspect the actual current homepage-related implementation and patch it rather than recreating it.

At minimum inspect:

- homepage-related contracts
- homepage-related DTOs
- existing homepage services/placeholders
- [HomepageSection](#)
- [HomepageSectionTranslation](#)

- `HomepageDraft`
- `PreviewToken`
- homepage seeders
- any homepage-related controllers or admin endpoints already present
- public homepage rendering implemented in PX02

You must build from current repo reality.

## 2. Implement or complete `HomepageSectionService`

Patch or complete the real homepage section service so it supports the fixed homepage model.

At minimum support:

- `getSections()`
- `getSectionByKey(string $key)`
- `updateSection(string $key, array $payload, string $locale)`
- `toggleSection(string $key, bool $enabled)`
- `reorderSections(array $orderedKeys)`
- `getPublicHomepage(string $locale)`
- `validateSectionPayload(string $key, array $payload, string $locale)`

Requirements:

- fixed key set only
- ordered homepage result
- locale-aware translation retrieval
- structured validation output
- no raw model returns publicly
- only published/enabled content should flow into the public homepage payload

## 3. Implement or complete `HomepagePublishingService`

Patch or complete the real homepage publishing workflow.

At minimum support:

- saving draft snapshots
- publishing now
- unpublishing
- scheduling future publish
- publish readiness validation
- audit logging
- cache invalidation

Requirements:

- draft content must never leak publicly
- required homepage content must be validated before publish
- publish must block if mandatory content is missing

- scheduling must preserve future publish intent cleanly
- unpublish must remove the homepage from public published state while preserving draft/history as appropriate

#### 4. Implement or complete homepage preview behavior

Patch or complete `PreviewService` as needed for homepage use.

At minimum support:

- homepage preview token issuance
- token TTL behavior
- locale-aware preview hydration
- desktop/tablet/mobile preview mode compatibility
- preview cache bypass compatibility

Preview rules:

- tokenized access only
- no public leakage of draft content
- preview should work for AR and EN independently
- preview output should align with the public homepage payload shape as much as practical

#### 5. Formalize the homepage section payload schemas

Implement clear structured payload expectations for each fixed section key.

You must support these sections explicitly and only:

`hero`

Support at minimum:

- background image required
- optional video
- overlay config
- headline
- subheadline
- primary CTA label + URL
- secondary CTA label + URL
- optional badge/kicker
- optional alignment config

`hero_stats`

Support at minimum:

- default 4 stat cards

- card fields such as:
  - value
  - optional suffix/prefix
  - label
  - optional icon
  - optional helper text
  - optional link
- ordering support
- frontend-safe display config if needed

## `academic_faculties`

Support at minimum:

- section title
- optional subtitle
- repeating faculty cards/items
- item fields such as:
  - title
  - short description
  - icon or image
  - accent/theme token
  - CTA label
  - CTA URL
- optional section CTA

## `achievements_highlights`

Support at minimum:

- section title
- optional subtitle
- repeating highlight cards
- item fields such as:
  - title
  - short text
  - optional icon
  - optional metric
  - optional date/label
  - CTA label
  - CTA URL

## `university_news`

Support at minimum:

- section title
- card collection
- manual selection mode and fallback shell mode

- item fields such as:
  - image
  - title
  - optional excerpt
  - publish date
  - category label
  - optional badge/tag
  - CTA URL
- section CTA label + URL

## research\_studies

Support at minimum:

- section title
- card collection
- manual selection mode and fallback shell mode
- item fields such as:
  - optional image
  - title
  - optional excerpt
  - publish date
  - category/type
  - optional authors
  - CTA URL
- section CTA label + URL

## events\_activities

Support at minimum:

- section title
- highlighted event/activity card(s)
- optional mini-calendar/highlighted dates payload
- item fields such as:
  - optional image
  - title
  - date
  - optional time
  - optional location
  - optional short description
  - CTA URL
- optional mobile/calendar config

## medical\_facilities\_services

Support at minimum:

- section title

- repeating service/facility cards
- item fields such as:
  - title
  - short description
  - image
  - CTA label
  - CTA URL
  - optional type tag

### **bottom\_stats**

Support at minimum:

- dark stats strip model
- default 4 stat items
- item fields such as:
  - numeric value
  - label
  - optional suffix/prefix
- ordering support

### **footer**

Support at minimum:

- logo/brand block
- contact block
- optional map/embed block
- social links
- footer navigation groups
- legal links
- copyright text
- optional emergency notice zone/config
- AR/EN independent contact/footer text

## 6. Keep payloads structured and editor-safe

All homepage section content must remain structured JSON/payloads.

Do not degrade the homepage into:

- raw unsanitized HTML blobs
- arbitrary untyped JSON
- old legacy block naming
- uncontrolled user-created section sets

## 7. Patch homepage seeders/factories only as needed

Review the current homepage-related seeders and patch them where needed so local development remains usable.

Requirements:

- preserve the fixed 10-key homepage model
- ensure AR and EN translation rows exist
- ensure payloads are realistic enough for local UI/runtime development
- ensure seeders remain suitable for local rebuilds
- do not pretend seeded placeholder content is final migration-ready public content

## 8. Integrate audit and cache behavior

Homepage write actions must go through the service layer and must:

- log audit actions
- invalidate appropriate homepage-related cache tags
- not flush unrelated public cache unnecessarily

At minimum ensure audit coverage for:

- homepage section update
- homepage draft save
- homepage publish
- homepage schedule
- homepage unpublish

## 9. Keep this phase strictly homepage-focused

This phase is only for completing homepage CMS behavior.

Do not expand into:

- full page-builder workflows beyond what homepage preview/publish needs
- full News/Event/Research/Facilities content modules
- full Filament UX completion
- redirect continuity or SEO continuity work outside homepage payload compatibility

## REQUIRED INPUTS

You must derive this phase from the actual current repository, including at minimum:

- homepage-related contracts in `app/Contracts`
- homepage-related DTOs in `app/DTOs`
- homepage services/placeholders in `app/Services`
- `app/Models/HomepageSection.php`
- `app/Models/HomepageSectionTranslation.php`
- `app/Models/HomepageDraft.php`
- `app/Models/PreviewToken.php`

- homepage-related migrations
- homepage seeders already in [database/seeders](#)
- current public homepage runtime from PX02
- current cache/audit services already present

You must build from the work already done.

## REQUIRED IMPLEMENTATION OUTCOMES

By the end of this phase, the homepage CMS should have these properties:

### Section management

- fixed 10-section model only
- locale-aware payload editing
- structured payload validation
- ordered section retrieval
- publish-safe public homepage payload retrieval

### Draft/publish workflow

- homepage draft save works
- homepage publish works
- homepage unpublish works
- homepage schedule works
- required-field validation blocks invalid publish

### Preview

- homepage preview token works
- preview respects locale
- preview is non-public
- preview integrates cleanly with cache bypass behavior

### Public compatibility

- public homepage runtime consumes published/enabled section data only
- AR and EN homepage payloads can differ independently

### Operational safety

- audit logs exist for homepage writes
- homepage cache invalidation exists and is targeted

## VERIFICATION REQUIREMENTS

At the end of the phase, verify at minimum:

- the 10 approved homepage section keys are the only active homepage keys
- `getSections()` returns the homepage sections in correct order
- each section can retrieve locale-specific payload
- homepage section update works per locale
- homepage preview token is issued and valid
- preview does not expose draft content publicly
- publish blocks if required homepage content is missing
- publish/unpublish/schedule go through the service layer
- homepage publish invalidates homepage cache
- homepage write actions create audit log entries
- public homepage payload hydrates correctly from published data only

## DONE WHEN

- The homepage CMS workflow is real, not placeholder behavior.
- Homepage sections are managed through a structured fixed-key service model.
- Draft/publish/schedule/unpublish flows work safely.
- Preview works safely and tokenized.
- Homepage public rendering now depends on a real published homepage state.
- The repository is ready for the next phase of shared shell/navigation/settings completion.

## CONSTRAINTS

- Do not reintroduce old legacy homepage keys.
- Do not model homepage sections as arbitrary user-created blocks.
- Do not degrade structured payloads into raw editor blobs.
- Do not build full downstream public modules here.
- Do not put publish/preview business logic into controllers or models.
- Do not return raw Eloquent models from public service methods.
- Patch and extend the current repo foundation rather than replacing it.

## ARCHITECTURE RULES

- Service Layer is the only place for business logic.
- Controllers remain thin.
- Models remain passive.
- Homepage payloads must stay structured and predictable.
- Public homepage rendering must depend on published/enabled state only.
- Use the existing repository implementation as the base truth.

## IMPORTANT IMPLEMENTATION NOTE

This phase assumes:

- foundation normalization is complete
- real public homepage runtime exists from PX02
- homepage schema and seed structure already exist in meaningful form

Do not behave as if the homepage CMS is being built from zero.

You are completing and normalizing an already-started homepage foundation.

## PX04-NAVIGATION-SETTINGS-SHELL — Finish Navigation, Utility Shell, Footer, and Settings on

Top of the Real Runtime

DEPENDS ON

PX02-PUBLIC-RUNTIME

PX03-HOMEPAGE-CMS

CONTEXT

Project: Syrian Private University website ([spu.edu.sy](https://spu.edu.sy))

Stack: Laravel 12, Filament v3, MySQL 8, Redis

Architecture rule: Service Layer is the only place for business logic

The repository already contains substantial shared-shell foundation pieces, including:

- [menu\\_items](#)
- [settings](#)
- seeded navigation and settings data
- imported navigation/settings-related legacy data
- existing contracts/DTOs/services for menu, navigation, settings, cache, and audit
- public runtime groundwork from PX02
- homepage completion from PX03

However, the shared public shell is not yet finished as a clean, stable system.

This phase is for completing the shared shell that every public page depends on:

- primary navigation
- utility navigation
- footer payload
- shared CTA/settings-driven links
- emergency notice behavior
- active-state resolution
- cache invalidation for shell-level writes

This phase is not for:

- redirect continuity
- sitemap
- [robots.txt](#)
- full module-specific content systems
- final launch validation
- complete Filament UX polish

GOAL

Complete the shared navigation/settings layer so the public homepage and landing-page shell can rely on a stable, service-driven source for:

- primary nav
- utility nav
- footer/legal/social/contact payloads
- language-switch metadata
- emergency notice state/content
- settings-backed CTA and external link targets
- active-state hints
- predictable cache invalidation

The result should eliminate shell-level placeholder behavior and make shared public layout data dependable.

## TASK

Implement or complete the shared shell layer using the current repository foundation.

You must do all of the following.

1. Review the existing navigation/settings foundation

Inspect the current repo implementation before patching anything.

At minimum inspect:

- `app/Contracts/MenuServiceInterface.php`
- `app/Contracts/NavigationServiceInterface.php`
- `app/Contracts/SettingsServiceInterface.php`
- related DTOs in `app/DTOs`
- existing menu/navigation/settings services or placeholders
- `app/Models/MenuItem.php`
- `app/Models/Setting.php`
- relevant cache and audit services
- `routes/web.php`
- any admin endpoints or seeders already touching menu/settings
- existing seeded and imported shell-related data already present in the DB

You must build from current repo reality.

2. Implement or complete `MenuService`

Patch or complete the real menu service.

At minimum support:

- create menu item
- update menu item
- delete menu item or disable item according to current product direction
- reorder menu tree
- toggle enabled state

- enforce max depth 2
- resolve supported target types
- fetch primary menu tree by locale
- fetch utility menu tree by locale
- support later admin consumption cleanly

#### Requirements:

- depth > 2 must be rejected at service level
- locale-aware menu retrieval must be stable
- utility vs non-utility behavior must be intentional
- service should return DTOs or structured payloads, not raw public model returns

Supported target behaviors should be consistent with current foundation, such as:

- page target
- custom URL
- external URL
- system route target if the current architecture still supports it

Do not invent wide-scope routing targets beyond what the current foundation needs.

### 3. Implement or complete `SettingsService`

Patch or complete the real settings service for the shared public shell.

At minimum support grouped and locale-aware handling for settings such as:

- apply CTA target
- student portal URL
- staff access URL
- language switcher config if settings-driven
- emergency notice state/content
- footer contact payload
- footer social payload
- legal/footer link payloads
- default SEO fallback settings where shared shell needs them

#### Requirements:

- support structured JSON/text settings appropriately
- support locale-aware reads where relevant
- support grouped retrieval for public shell consumption
- support update flows for later admin use
- invalidate the correct cache groups after writes

You must use the current repo's actual `settings` table structure and existing group/key conventions where possible, patching only where needed.

#### 4. Implement or complete `NavigationService`

Patch or complete the real navigation aggregation service.

At minimum support:

- `getHeaderNavigation(string $locale, ?string $currentPath = null)`
- `getFooterNavigation(string $locale)`
- `getUtilityNavigation(string $locale)`
- `getFullNavigationPayload(string $locale, ?string $currentPath = null)`

The navigation payload should combine:

- primary navigation
- utility navigation
- language-switch metadata
- CTA/settings-driven links
- emergency notice state/content where relevant
- active-state hints for current route/path
- footer navigation/footer settings payload

Requirements:

- active state must be derived consistently
- locale-specific URLs must be correct
- navigation payload shape must be reusable across homepage and landing-page shell rendering
- avoid coupling controllers/views to raw menu/settings logic

#### 5. Finalize footer shell behavior

Complete the footer shell payload so it can be rendered cleanly by the public runtime.

At minimum support:

- brand/logo block references
- footer navigation groups
- legal links
- contact information
- social links
- optional map/embed references if current foundation supports them
- locale-aware footer text/content
- emergency notice or footer notice integration if current product direction requires it

Do not treat footer data as ad hoc scattered settings.

It should be resolved into a coherent payload through services.

#### 6. Finalize utility navigation behavior

Utility shell behavior must be stable and intentional.

At minimum handle:

- apply CTA
- student portal URL
- staff access URL
- language switch context
- any emergency/announcement utility behavior the current shell expects

Requirements:

- utility shell data must not be hardcoded in views
- utility items must be locale-aware where needed
- settings-backed utility values must be consistently exposed through one service path

7. Ensure active-state resolution works correctly

Implement or patch active-state behavior in a clean, reusable way.

At minimum:

- current page/path should mark appropriate nav item active
- locale-prefixed URLs must still resolve active state correctly
- parent item active behavior should be consistent
- utility and footer behavior should not incorrectly mark unrelated items active

Do not put active-state logic into Blade templates unless truly trivial.

8. Integrate cache invalidation correctly

This phase must integrate shell-level cache invalidation properly.

At minimum ensure:

- menu changes invalidate navigation cache
- settings changes invalidate settings cache
- settings changes that affect navigation also invalidate navigation cache
- footer/utility changes invalidate the right shared shell payloads
- shell cache invalidation is targeted, not global by default

Use the current repo's cache service and tag strategy where available, patching as needed.

9. Integrate audit logging for shell writes

Menu/settings shell write actions must be auditable.

At minimum support audit logging for:

- menu created
- menu updated
- menu reordered
- menu toggled
- menu deleted/disabled
- settings updated
- utility-shell-affecting setting changes
- footer-affecting setting changes

Use the current audit service foundation rather than scattering write logs manually.

#### 10. Patch shell-related seed assumptions only where needed

Review existing seeders and current seeded shell data only as needed to support a coherent shell foundation.

Goals:

- preserve useful local/dev shell scaffolding
- avoid pretending placeholder nav/settings are final launch-ready IA/content
- make sure shell seed defaults remain compatible with the real service payloads
- keep local rebuilds usable

Do not turn this phase into a full IA redesign.

#### 11. Keep this phase limited to shared shell behavior

This phase is only for the shared public shell and its supporting service logic.

Do not expand into:

- redirect continuity
- sitemap/robots
- full SEO continuity
- full admin/Filament completion
- full module CRUD or public module delivery
- admissions/contact workflow expansion beyond shared shell links/settings

### REQUIRED INPUTS

You must derive this phase from the actual current repository, including at minimum:

- menu/navigation/settings contracts in `app/Contracts`
- relevant DTOs in `app/DTOs`
- menu/navigation/settings services/placeholders in `app/Services`
- `app/Models/MenuItem.php`
- `app/Models/Setting.php`
- cache and audit services
- `routes/web.php`

- public views/layouts created in PX02
- homepage public runtime from PX03
- existing menu/settings seeders
- existing imported menu/settings data already present in the DB

You must build from the work already done.

## REQUIRED IMPLEMENTATION OUTCOMES

By the end of this phase, the shared shell should have these properties:

### Navigation

- primary nav is real and service-driven
- utility nav is real and service-driven
- locale-aware nav URLs work
- active-state hints are included
- depth > 2 is rejected

### Settings

- shared shell settings are grouped and readable through one coherent service path
- utility/footer/emergency/shared-link settings can be resolved reliably
- locale-aware behavior exists where needed

### Footer

- footer payload is coherent and renderable
- social/contact/legal/footer-group data comes from real services/settings

### Operational behavior

- shell writes invalidate the right cache groups
- shell writes create audit logs
- shell payload is reusable across homepage and landing-page shell rendering

## VERIFICATION REQUIREMENTS

At the end of the phase, verify at minimum:

- primary navigation payload resolves correctly for [ar](#) and [en](#)
- utility navigation payload resolves correctly for [ar](#) and [en](#)
- footer payload resolves correctly for [ar](#) and [en](#)
- menu depth > 2 is rejected
- current path can mark active nav items correctly
- apply CTA/student portal/staff access values resolve through services/settings
- emergency notice payload resolves correctly if configured
- menu updates invalidate navigation cache

- settings updates invalidate affected settings/navigation cache
- shell write actions create audit log rows

## DONE WHEN

- The shared navigation/settings/footer shell is real, coherent, and service-driven.
- Homepage and landing-page shell rendering can depend on stable navigation and settings payloads.
- Utility/footer/CTA/emergency/shared-link behavior is no longer placeholder-driven.
- Menu and settings writes are auditable and cache-safe.
- The repository is ready for the next phase of SEO output and migration continuity work.

## CONSTRAINTS

- Do not implement redirect continuity in this phase.
- Do not implement sitemap in this phase.
- Do not implement `robots.txt` in this phase.
- Do not expand into full module feature delivery.
- Do not push menu/settings logic into controllers or Blade templates.
- Do not return raw Eloquent models from public service methods.
- Patch and extend the current repo foundation rather than replacing it.

## ARCHITECTURE RULES

- Service Layer is the only place for business logic.
- Controllers remain thin.
- Models remain passive.
- Navigation/footer/settings payloads must be built through services.
- Shared shell behavior must be cache-aware and audit-aware.
- Use existing schema and current repository work as the base truth.

## IMPORTANT IMPLEMENTATION NOTE

This phase assumes:

- PX02 gave the project a real public runtime
- PX03 completed homepage CMS behavior
- menu/settings schema and some shell data already exist in the repository

Do not behave as if navigation/settings are being built from zero.

You are completing and normalizing an already-started shared shell foundation.

---

## PX05-SEO-CONTINUITY — Implement SEO Output, Redirect Continuity, Sitemap, Robots, and File Continuity

DEPENDS ON

PX02-PUBLIC-RUNTIME

PX04-NAVIGATION-SETTINGS-SHELL

CONTEXT

Project: Syrian Private University website ([spu.edu.sy](http://spu.edu.sy))

Stack: Laravel 12, Filament v3, MySQL 8, Redis

Architecture rule: Service Layer is the only place for business logic

This is now one of the true launch-blocking phases.

Current repo reality that must be treated as true:

- SEO-related schema already exists:
  - [page\\_seo\\_meta](#)
  - settings-backed SEO defaults
  - page/media structures that can support OG/canonical behavior
- public homepage and landing-page runtime should already exist from prior phases
- shared shell/navigation/settings should already exist from prior phases
- legacy import audit/export tooling already exists and has been used
- migration audits have already identified the following as major gaps:
  - no redirect continuity layer
  - no sitemap
  - no [robots.txt](#)
  - no public canonical rendering
  - no public hreflang rendering
  - no file continuity layer
  - unresolved legacy URL/file continuity risk
- legacy imports and audit data already exist and should be used as inputs rather than ignored
- this phase must be grounded in actual repository and import reality, not generic SEO theory

This phase is for implementing the real SEO output and continuity layer required before any serious migration/cutover can be considered.

This phase is not for:

- full public modules outside the homepage/admin foundation
- final Filament completion
- final launch checklist execution
- replacing already-audited import logic without cause

## GOAL

Implement the real continuity and SEO layer required for migration readiness, including:

- canonical tags
- hreflang tags
- meta title/description output
- OG output
- robots directives
- sitemap generation
- [robots.txt](#)
- redirect continuity
- unresolved-request logging
- file/PDF continuity support

The result should move the project from “SEO fields exist in storage” to “SEO and continuity are actually working in runtime.”

## TASK

Implement the SEO output and continuity layer on top of the current repository foundation.

You must do all of the following.

1. Review the current SEO and continuity baseline already in the repo

Inspect the actual current implementation and data model before building anything.

At minimum inspect:

- [page\\_seo\\_meta](#) migration/model
- page service and SEO-related contracts/DTOs/services
- current public homepage/page controllers/views from prior phases
- current settings-backed SEO defaults
- imported page SEO/meta data if already present
- current menu/page/locale route structure
- current cache behavior
- current legacy import audit/export tooling
- current migration guide and SEO audit conclusions already available in the repo/workspace context

You must derive this phase from current repo reality.

2. Implement real SEO rendering for homepage and landing pages

Patch or complete the runtime SEO rendering layer.

At minimum support:

- `<title>`
- meta description
- canonical URL
- robots tag when set
- OG title
- OG description
- OG image where available
- locale-aware SEO hydration for homepage and landing pages
- settings-backed SEO fallback when page-specific metadata is incomplete

#### Requirements:

- output must be driven through the service layer
- views must receive structured SEO payloads
- canonical URLs must be absolute
- AR/EN rendering must be correct and intentional
- no ad hoc SEO logic should live directly in controllers

### 3. Implement hreflang output and validation behavior

Patch or complete hreflang support for public pages.

At minimum support:

- AR hreflang
- EN hreflang
- reciprocal locale mapping where equivalent page/home content exists
- locale-aware alternate links in the rendered HTML
- homepage hreflang support
- landing-page hreflang support

#### Requirements:

- hreflang output must be generated from real routing/page context
- invalid or non-reciprocal mappings should not silently produce misleading tags
- if current schema stores hreflang payload, validate or normalize usage rather than blindly trusting it
- keep behavior aligned with the actual locale strategy already chosen by the repo/project

### 4. Implement real canonical URL resolution

Patch or complete canonical generation through the service layer.

At minimum support:

- homepage canonical for `/ar` and `/en`
- landing-page canonical per locale and slug
- settings/domain-aware absolute canonical generation
- fallback canonical generation if explicit canonical is missing

- no accidental cross-locale canonical collisions

#### Requirements:

- canonical logic must use current runtime routing reality
- canonical logic must not depend on hardcoded temporary host assumptions where settings or config should own them
- canonical generation must be deterministic and testable

### 5. Implement sitemap generation

Create or complete sitemap support for the current foundation scope.

At minimum support:

- homepage URLs
- landing-page shell URLs
- locale-aware output
- only publicly visible/published pages
- XML responses suitable for search engines

You may choose:

- one sitemap
- sitemap index + child sitemaps

Choose the smallest correct approach for the current foundation scope.

#### Requirements:

- do not include draft/unpublished/scheduled-not-yet-public pages
- do not include admin URLs
- do not include preview URLs
- locale-aware URLs must be correct
- output must reflect actual public runtime state

### 6. Implement `robots.txt`

Create or complete runtime support for `robots.txt`.

#### Requirements:

- must reflect current environment
- production intent should be indexable only where appropriate
- non-production/staging-like environments should be safely controllable
- reference sitemap location(s)
- avoid generic placeholder robots behavior that conflicts with the real route layout

Do not hardcode an unsafe production indexing policy without considering environment/config.

## 7. Implement redirect continuity schema

Add the database foundation needed for continuity.

Create migrations as needed for a DB-backed continuity layer, at minimum covering:

- exact legacy redirects
- pattern-based redirect rules
- unresolved legacy requests
- file continuity inventory
- optional legacy URL mapping/reference table if your chosen design needs it

A typical shape may include tables like:

- `legacy_exact_redirects`
- `legacy_pattern_rules`
- `unresolved_legacy_requests`
- `legacy_file_inventory`
- `legacy_url_map` if needed

Use names that fit the existing repo conventions, but the resulting architecture must support real continuity work.

Requirements:

- schema must be migration-safe and reversible
- schema must be usable by a service layer, not just by ad hoc SQL
- schema must support auditability/debugging of unresolved requests

## 8. Implement redirect continuity runtime

Create or complete the redirect continuity layer in runtime.

At minimum support:

- exact legacy URL redirects
- pattern/query-string-aware legacy redirects where needed
- locale-aware destination resolution where possible
- safe HTTP status handling
- unresolved-request logging when no rule matches
- no redirect loops
- no redirect logic scattered across unrelated controllers

Requirements:

- continuity logic must be service-driven and/or middleware-driven
- matching priority must be explicit and deterministic
- unresolved legacy requests must be observable
- redirect behavior must account for current `/admin` path reality

- continuity should support future bulk import of redirect maps

## 9. Implement unresolved legacy request logging

When a legacy URL cannot be resolved, the app must log it in a structured way rather than failing silently.

At minimum capture:

- requested URL/path
- query string
- method
- referrer if available
- resolved locale if any
- timestamp
- optional user-agent/IP if consistent with app policy
- whether the request appears to be file-like or page-like

Requirements:

- unresolved logging must be queryable later
- logging should not create runaway duplicate spam without thought
- do not log through ad hoc text files when the app already has structured DB-backed approaches available

## 10. Implement file/document continuity support

Create or complete the continuity mechanism for old files, PDFs, and document-like assets.

At minimum support:

- inventory of known legacy file paths or references
- mapping from legacy file paths to current delivery paths or media assets
- runtime resolution for supported file continuity requests
- observability for unresolved file requests

Requirements:

- use actual repo import data and media structures where possible
- do not assume all files are already publicly reachable
- be mindful that `public/storage` status and public file delivery are part of real runtime behavior
- file continuity should be testable and not rely on undocumented manual server behavior

## 11. Reuse the existing import audit/export infrastructure

This phase must use the migration-readiness work already present in the repo.

Leverage existing commands and data such as:

- `legacy-import:report`
- `legacy-import:verify`
- `legacy-import:audit`
- `legacy-import:export-missing`
- `migration_logs`
- `migration_rejections`
- `legacy_record_snapshots`

Use them where they can help:

- seed redirect/file continuity input
- classify unresolved legacy URL/file issues
- identify imported page/file candidates for continuity mapping

Do not ignore this existing work and rebuild continuity assumptions from scratch.

## 12. Integrate cache invalidation and runtime safety

SEO/continuity changes must play correctly with the existing caching strategy.

At minimum ensure:

- SEO-related page changes invalidate affected page/SEO cache
- sitemap output does not serve stale or invalid page state
- preview routes remain bypassed
- unresolved logging does not break public runtime
- redirect matching is efficient enough for the current scope

## 13. Keep this phase limited to SEO + continuity

This phase is only for the SEO and migration continuity layer needed by the homepage/admin foundation and public landing-page shell.

Do not expand into:

- full cross-site search
- full legacy content module replacement
- full archive-site implementation
- full analytics/marketing platform integrations
- final admin/Filament UX refinement
- final launch checklist execution

## REQUIRED INPUTS

You must derive this phase from the actual current repository, including at minimum:

- `app/Contracts/*` related to SEO, pages, settings, preview, cache
- `app/DTOs/*` related to page SEO/navigation/page payloads
- `app/Services/*` related to SEO, page, settings, cache, preview

- `app/Models/Page.php`
- `app/Models/PageSeoMeta.php`
- `app/Models/Setting.php`
- `app/Models/MediaAsset.php`
- existing public controllers/routes/views from prior phases
- existing cache service
- existing audit/import/export commands in `routes/console.php`
- import-related DB tables already present
- migration guide/SEO audit findings already available in repo/workspace context

You must build from actual work already done.

## REQUIRED IMPLEMENTATION OUTCOMES

By the end of this phase, the project should have these properties:

### Public SEO output

- canonical tags render on homepage and landing pages
- hreflang tags render on homepage and landing pages
- meta title/description render correctly
- OG tags render correctly
- robots directives render when set
- settings-backed fallbacks work intentionally

### Sitemap and robots

- sitemap endpoint(s) exist and are valid
- `robots.txt` exists and is environment-aware
- only public/published URLs are exposed

### Redirect continuity

- exact and pattern-based continuity is possible
- unresolved legacy requests are logged structurally
- continuity does not depend on scattered ad hoc logic

### File continuity

- old file/document continuity has a real runtime foundation
- unresolved file requests are observable
- continuity works with current media/storage architecture

## VERIFICATION REQUIREMENTS

At the end of the phase, verify at minimum:

- homepage renders canonical + hreflang correctly for `ar` and `en`
- at least one landing page renders canonical + hreflang correctly

- OG tags render when data exists
- settings fallback SEO works when page SEO is incomplete
- sitemap endpoint returns valid XML
- sitemap excludes draft/unpublished/admin/preview URLs
- `robots.txt` returns a valid environment-appropriate response
- exact legacy redirect resolution works for representative cases
- unresolved legacy request logging works for unmatched paths
- file/document continuity works for at least representative mapped cases
- unresolved file requests are logged structurally
- continuity and SEO-related cache behavior remains correct

## DONE WHEN

- SEO metadata is no longer only stored, but actually rendered.
- Canonical and hreflang behavior is live and intentional.
- Sitemap and `robots.txt` exist and reflect real public state.
- Redirect continuity has a real schema and runtime layer.
- Unresolved legacy requests are logged structurally.
- File/document continuity has a real foundation.
- The repository is ready for the next phase of admin/Filament completion and migration backfill tooling.

## CONSTRAINTS

- Do not build full out-of-scope public modules.
- Do not scatter redirect logic across unrelated controllers.
- Do not treat stored SEO fields as sufficient without runtime rendering.
- Do not hardcode unsafe host/canonical assumptions where config/settings should own them.
- Do not bypass the existing service-layer architecture.
- Do not ignore the existing import audit/export infrastructure already present in the repo.

## ARCHITECTURE RULES

- Service Layer is the only place for business logic.
- Controllers remain thin.
- Models remain passive.
- Continuity should be service- and/or middleware-driven.
- SEO output should be generated through services and structured view payloads.
- Use current repo schema and prior audited work as the base truth.

## IMPORTANT IMPLEMENTATION NOTE

This phase assumes:

- public homepage and landing-page runtime already exist from earlier phases
- shared shell/navigation/settings behavior already exists from earlier phases
- SEO/storage/import foundations already exist in the repository

- migration continuity is currently missing and must now be added properly

Do not behave as if SEO continuity is being built in a vacuum.

You are implementing the missing runtime layer on top of already-existing storage, import, and content foundations.

## PX06-ADMIN-FILAMENT-COMPLETION — Complete Admin Controllers and Filament UI on Top of Real Services

### DEPENDS ON

PX03-HOMEPAGE-CMS

PX04-NAVIGATION-SETTINGS-SHELL

PX05-SEO-CONTINUITY

### CONTEXT

Project: Syrian Private University website ([spu.edu.sy](https://spu.edu.sy))

Stack: Laravel 12, Filament v3, MySQL 8, Redis

Architecture rule: Service Layer is the only place for business logic

The repository already contains meaningful admin foundation work, including:

- Filament mounted under [/admin](#)
- auth/RBAC foundation
- users/roles schema
- audit logging schema
- page/homepage/menu/settings/media schema
- service contracts and partial implementations
- public runtime and continuity work completed in prior phases of this rebuilt plan

This phase is not greenfield admin creation.

It is for completing the admin experience on top of the real services and runtime already implemented.

Current repo reality that must be treated as true:

- Filament already exists
- [/admin](#) is already active
- early admin/auth scaffolding already exists
- placeholder service usage may still exist in some admin/resource paths and must be replaced with real service-backed behavior
- homepage/page/menu/settings/media/user/audit areas already have schema-level support
- this phase should not re-architect the public runtime
- this phase should not implement full out-of-scope public modules

This phase is for:

- finishing admin controllers where needed
- completing Filament pages/resources
- connecting the admin UX to real services
- making preview/draft/publish/menu/settings/media/user flows usable and safe

This phase is not for:

- full site-wide content modules
- launching the site
- replacing continuity architecture
- rebuilding auth from scratch

## GOAL

Complete the admin panel and Filament layer so non-technical staff can safely manage the homepage/admin foundation through real service-backed workflows.

This includes:

- homepage editing
- landing-page editing
- menu builder
- media library
- settings management
- preview actions
- publish/schedule/unpublish actions
- user management
- audit log visibility
- role-based resource visibility

The result should be a working admin UI built on real services, not placeholder logic.

## TASK

Implement or complete the admin controller and Filament layer using the current repository foundation.

You must do all of the following.

1. Review the current admin and Filament foundation

Inspect the current repository implementation before patching anything.

At minimum inspect:

- Filament panel/provider configuration
- existing admin routes/controllers
- existing Filament resources/pages/widgets
- auth/RBAC gates/policies
- current admin middleware and route protection
- current service bindings and any remaining placeholders
- homepage/page/menu/settings/media/user/audit models and services
- public preview/runtime contracts and services from previous phases

You must build from current repo reality and patch it, not regenerate blindly.

## 2. Complete admin controllers where standard Laravel controllers are still needed

Patch or create standard admin controllers only where they still belong outside Filament resource/page classes.

At minimum, support the admin foundation areas as needed:

- homepage actions
- page actions
- menu actions
- media actions
- settings actions
- preview actions
- user actions
- audit read endpoints where appropriate

### Requirements:

- all admin routes must remain protected by admin auth and authorization
- controllers must use interfaces/services only
- no raw model business logic in controllers
- JSON responses for reorders/toggles/uploads/preview-token issuance where appropriate
- redirect/view responses only where appropriate
- all admin writes must go through services
- admin writes must generate audit log entries
- admin writes must invalidate relevant cache groups where applicable

If the current repo already shifted some of these responsibilities fully into Filament, patch only what is still needed and avoid duplication.

## 3. Complete homepage admin editing UX

Implement or complete a real homepage builder/admin editing surface.

This can be a Filament page or the repo's chosen equivalent, but it must be built on the real homepage services already completed in prior phases.

### Requirements:

- represent the homepage as the fixed 10-section model only
- support AR and EN editing separately
- allow section-level editing using structured forms
- allow section enable/disable where permitted
- allow homepage draft save
- allow homepage preview
- allow homepage publish
- allow homepage schedule

- allow homepage unpublish
- display validation errors clearly
- display state clearly

Do not expose raw JSON as the primary editing experience unless clearly justified for advanced/internal use only.

#### 4. Complete landing-page admin editing UX

Implement or complete the landing-page management/admin editing surface.

Requirements:

- manage top-level and child landing pages created by the page builder/runtime foundation
- support base metadata
- support hierarchy/parent assignment
- support slug editing according to the chosen URL model
- support status visibility
- support enabled/nav/breadcrumb toggles
- support last reviewed date if present
- support AR translation payload editing
- support EN translation payload editing
- support AR SEO payload editing
- support EN SEO payload editing
- support save draft
- support preview
- support publish
- support schedule
- support unpublish
- support soft delete/restore only if the current repo direction supports it intentionally

Requirements:

- use structured field groups/tabs/sections
- keep editor experience manageable for AR and EN
- rely on the real page service and SEO service
- do not embed business rules directly in resource/forms

#### 5. Complete menu builder UX

Implement or complete the admin/Filament menu builder.

Requirements:

- manage primary navigation and utility navigation
- support nesting up to depth 2 only
- support drag/drop or deterministic ordering workflow

- support page/custom/external/system-route targets as actually supported by the current architecture
- support locale-aware editing where required
- support enable/disable state clearly
- support utility grouping clearly
- use the real menu/navigation services
- surface validation errors when depth or target rules are violated

## 6. Complete media library/admin media UX

Implement or complete the media admin layer.

Requirements:

- upload files through the real media service
- list/search/filter media
- edit title/alt/caption in AR and EN
- display previews where practical
- expose useful metadata such as:
  - public URL
  - file type
  - dimensions
  - generated WebP/srcset data if available
- support delete/soft-delete behavior according to the current repo's media policy
- surface upload validation errors clearly

Do not bypass the media service by writing upload logic directly inside Filament resource classes.

## 7. Complete settings admin UX

Implement or complete the admin settings experience for the shared shell and SEO defaults.

At minimum support settings groups such as:

- utility navigation
- footer
- emergency notice
- contact
- social
- SEO defaults
- shared shell/public settings

Requirements:

- use structured forms, not raw JSON as the default editor experience
- support locale-aware settings where relevant
- use the real settings service
- invalidate appropriate cache after writes
- audit settings changes

## 8. Complete user management UX

Implement or complete user management for the current admin foundation scope.

Requirements:

- super admin only
- manage role
- manage name/email
- manage password reset/update path as appropriate
- manage faculty scope slug if still relevant
- manage lock/unlock state
- avoid casually exposing dangerous fields
- rely on the real auth/RBAC/user foundation

Editors and other non-authorized roles must not gain access.

## 9. Complete audit log UX

Implement or complete the audit log read experience.

Requirements:

- read-only
- searchable/filterable where practical
- at minimum filterable by:
  - action
  - entity type
  - user
  - date/time range
- super admin only unless the current repo policy intentionally broadens read access

This should be built on the real audit data already being generated.

## 10. Complete preview actions in admin

Implement or complete admin preview issuance and preview launch behavior.

Requirements:

- homepage preview actions
- landing-page preview actions
- locale-aware preview
- device-mode support where prior phases exposed it
- safe preview-token issuance
- no public leakage
- admin UI should be able to launch preview cleanly

Do not create a fake preview layer separate from the real preview service.

## 11. Ensure role-based visibility and authorization are correct

Patch and complete role-aware access for the admin UI.

Requirements:

- super admin can access all foundation admin areas
- editor can access only allowed content/shell areas
- faculty\_editor remains restricted according to current project rules and scope
- forbidden resources/pages/actions must not merely fail late; visibility should also be cleaned up where appropriate
- use real gates/policies/services already present in the repo

## 12. Remove remaining placeholder admin behavior

Review the admin layer for places where placeholder services or stub data flows are still used.

Patch them so the admin panel is backed by the real services implemented in prior phases.

Do not leave Filament resources/pages depending on placeholder business logic if a real implementation now exists.

## 13. Keep this phase limited to the homepage/admin foundation

This phase is only for completing the admin layer for the current foundation scope.

Do not expand into:

- full public modules outside scope
- full CRM workflows
- full news/events/research/facilities editorial systems
- site-wide analytics/reporting dashboards unrelated to the foundation
- launch validation or rollback preparation

## REQUIRED INPUTS

You must derive this phase from the actual current repository, including at minimum:

- Filament config/panel/provider setup
- existing admin routes/controllers
- existing Filament resources/pages/widgets
- `app/Contracts/*`
- `app/DTOs/*`
- `app/Services/*`
- `app/Models/*` for homepage/page/menu/settings/media/user/audit
- auth/RBAC gates/policies/middleware
- public runtime/preview/services already completed in prior phases
- cache/audit/media/settings/page/homepage services
- current route structure under `/admin`

You must build from actual work already done in the repo.

## REQUIRED IMPLEMENTATION OUTCOMES

By the end of this phase, the admin layer should have these properties:

### Homepage admin

- homepage sections editable through structured UI
- homepage draft/preview/publish/schedule/unpublish flows work
- validation and state feedback are clear

### Landing-page admin

- pages editable through structured AR/EN editor
- SEO editable per locale
- draft/preview/publish/schedule/unpublish flows work

### Menu/settings/media admin

- menu builder is usable and enforces depth rules
- settings UI is usable and service-backed
- media library is usable and service-backed

### User/audit admin

- user management works for super admin only
- audit log read experience is available and useful
- role visibility is clean and correct

### Operational integrity

- admin writes go through services
- admin writes generate audit logs
- relevant cache invalidation occurs
- no key admin path still depends on placeholder business behavior

## VERIFICATION REQUIREMENTS

At the end of the phase, verify at minimum:

- Filament/admin loads without resource registration errors
- homepage editing UI works and uses real homepage services
- landing-page editing UI works and uses real page/SEO services
- menu builder enforces depth  $\leq 2$
- settings updates work and invalidate affected cache
- media upload/edit flows work through the media service
- user management is accessible only to authorized roles

- audit log resource/page is read-only and accessible only to authorized roles
- preview actions work for homepage and landing pages
- editor/super\_admin visibility differences are correct
- no major admin path still depends on placeholder business logic

## DONE WHEN

- The admin and Filament layer is fully connected to real services.
- Homepage/page/menu/settings/media/user/audit flows are usable and authorization-safe.
- The admin panel supports the current homepage/admin foundation without placeholder dependencies.
- The repository is ready for the next phase of migration backfill/tooling completion.

## CONSTRAINTS

- Do not rebuild Filament/admin from scratch if meaningful structure already exists.
- Do not put business logic into Filament resources/pages/forms.
- Do not expand into full out-of-scope public module management.
- Do not create parallel admin workflows that duplicate existing real services.
- Do not return raw Eloquent models from public-facing service contracts.

## ARCHITECTURE RULES

- Service Layer is the only place for business logic.
- Controllers and Filament classes must remain orchestration/UI layers.
- Models remain passive.
- Admin actions must go through services.
- Audit and cache invalidation responsibilities must remain in services or clearly centralized supporting layers.
- Use current repository work as the base truth.

## IMPORTANT IMPLEMENTATION NOTE

This phase assumes:

- public runtime is already real
- homepage CMS is already real
- shared shell/navigation/settings are already real
- SEO/continuity foundation is already in place

Do not behave as if the admin panel is being created from zero.

You are completing and normalizing the admin/Filament layer on top of an already-built repository foundation.

## PX07-MIGRATION-BACKFILL — Add URL Inventory, Redirect Import, File Continuity, and Reconciliation Tooling

DEPENDS ON

PX05-SEO-CONTINUITY

PX06-ADMIN-FILAMENT-COMPLETION

CONTEXT

Project: Syrian Private University website ([spu.edu.sy](http://spu.edu.sy))

Stack: Laravel 12, Filament v3, MySQL 8, Redis

Architecture rule: Service Layer is the only place for business logic

The repository already contains meaningful migration/import foundations, including:

- legacy import seeders
- [migration\\_logs](#)
- [migration\\_rejections](#)
- [legacy\\_record\\_snapshots](#)
- import report/audit/export commands already defined in [routes/console.php](#)
- audited import fixes and missing-data export behavior
- continuity schema/runtime from PX05 should already exist by the time this phase begins

This phase is not for generic legacy data exploration from scratch.

This phase is for converting the existing migration/import foundation into a practical [cutover-preparation](#) toolkit.

That means:

- exporting legacy URL inventories
- validating redirect maps before import/use
- generating file/document continuity inventories
- reporting unresolved/missing/ambiguous legacy mappings
- reconciling known legacy data ambiguities
- validating imported SEO/page continuity assumptions
- producing repeatable operational outputs instead of manual SQL-only workflows

Current repo reality that must be treated as true:

- import audit/export commands already exist and should be reused where possible
- imported data and missing-data inventories already exist from prior audit work
- legacy continuity cannot depend on ad hoc one-off SQL forever
- unresolved legacy URL/file mappings must become explicit, versionable, and repeatable
- migration readiness depends on tooling, not just schema

This phase is not for:

- final launch/cutover execution
- final rollback execution
- full-site archive implementation
- replacing already-working importers without cause
- expanding into out-of-scope full public modules

## GOAL

Add the tooling and reporting layer needed to move from:

- “legacy import data exists” to:
- “engineering can prepare, validate, and track migration continuity safely”

The result should make these things repeatable:

- legacy URL inventory export
- redirect map validation/import preparation
- file continuity inventory
- unresolved continuity reporting
- imported-content continuity validation
- reconciliation reporting for ambiguous legacy structures

## TASK

Implement the migration backfill and continuity tooling layer using the current repository foundations.

You must do all of the following.

1. Review the current migration/import tooling already in the repo

Inspect the existing repo tooling and build on it rather than ignoring it.

At minimum inspect:

- `routes/console.php`
- import-related seeders
- `migration_logs`
- `migration_rejections`
- `legacy_record_snapshots`
- any legacy import support services
- existing missing-data export logic
- any current SEO/static-page import seeders
- any current continuity tables/services from PX05

You must build from current repo reality and reuse existing infrastructure where appropriate.

2. Add a legacy URL inventory export workflow

Create or complete a command/tooling path that exports the legacy public URL inventory needed for redirect and continuity planning.

The export should be practical for engineering and migration operations.

At minimum support:

- legacy source table references where URLs can be inferred
- structured export of candidate legacy URLs
- classification of URL types where practical
- output suitable for later redirect mapping validation/import

A typical export should capture things like:

- source type/module
- legacy path
- legacy query string or normalized query signature if relevant
- expected destination candidate if already known
- locale if inferable
- status/classification fields
- source identifiers for traceability

Do not assume one legacy URL shape if the old site has multiple patterns.

### 3. Add redirect map validation tooling

Create or complete tooling that can validate redirect continuity input before it is trusted.

This should work with the continuity architecture implemented in PX05.

At minimum support:

- validating exact redirect rows
- validating pattern-rule inputs
- detecting duplicate/conflicting legacy source rules
- detecting missing destination data where required
- detecting obvious loops or invalid targets
- reporting invalid rows clearly

If the project direction supports importable redirect workbooks/CSV/JSON, validate those inputs before use.

The validator should help engineering answer:

- is this redirect map internally consistent?
- is it safe to load?
- what still needs manual resolution?

### 4. Add legacy file/document inventory tooling

Create or complete tooling that exports and/or reconciles legacy file/document references for continuity.

At minimum support:

- extracting legacy file/document-like references from current import sources or imported media traces
- reporting mapped vs unmapped legacy file references
- associating legacy file references to current media assets where known
- producing a machine-readable output for continuity review

This must be grounded in the repo's actual current import/media structures, not a hypothetical file model.

#### 5. Add unresolved continuity reporting

Create or complete commands/reports that make unresolved migration continuity visible.

At minimum report on:

- unresolved legacy URL requests
- unresolved file/document requests
- imported content still missing continuity-critical fields
- continuity mappings still requiring manual intervention

This reporting should be good enough for engineering and migration ops to track what remains before cutover.

#### 6. Add imported page SEO/continuity validation tooling

Create or complete tooling that validates whether imported and/or scaffolded page content is continuity-ready.

At minimum support reporting on issues such as:

- missing canonical data
- invalid canonical format
- missing or inconsistent hreflang mappings
- imported pages that cannot be reached through the current public URL model
- suspicious slug collisions or continuity mismatches
- imported legacy pages with weak or incomplete SEO metadata

This phase should help identify whether imported landing/static content is actually ready to participate in continuity.

#### 7. Add reconciliation reporting for ambiguous legacy structures

The legacy system contains known areas of ambiguity and collision.

Create or complete reporting for ambiguous or overlapping structures that require engineering review.

At minimum consider areas already observed in repo work such as:

- overlapping or competing legacy config sources
- overlapping council/member source behavior
- content imported from multiple source tables that could imply duplicates
- unresolved/non-AR-EN content handling
- legacy rows preserved only as snapshots and still needing final product treatment

The goal is not to solve every ambiguity automatically.

The goal is to make ambiguity visible, repeatable, and actionable.

8. Reuse and extend existing import audit/export commands where appropriate

Do not create redundant tooling if the repo already has a useful base.

Where appropriate, extend or complement:

- `legacy-import:report`
- `legacy-import:verify`
- `legacy-import:audit`
- `legacy-import:export-missing`

If new commands are needed, make them consistent with the existing console tooling style.

9. Produce machine-readable outputs suitable for handoff

Where this phase exports inventories or reports, they should be suitable for real engineering/ops handoff.

Prefer outputs such as:

- JSON
- CSV
- clearly structured command output

Examples of useful outputs:

- legacy URL inventory
- redirect validation report
- file continuity inventory
- unresolved continuity report
- imported page SEO readiness report
- ambiguity/reconciliation report

Do not make this phase depend only on reading console tables manually.

## 10. Keep continuity tooling grounded in actual current architecture

This phase must reflect:

- the current route structure
- the chosen locale strategy
- the chosen page URL model
- the redirect continuity schema added in PX05
- the current import/rejection/snapshot model
- the actual current media/file architecture

Do not build tooling around outdated assumptions from the original greenfield prompt pack.

## 11. Keep this phase operational, not speculative

This phase is for real migration-preparation tooling.

Do not expand into:

- full archive site implementation
- final cutover execution
- final rollback execution
- speculative full-site search migration
- broad analytics/reporting unrelated to continuity readiness
- rewriting already-correct importer logic without evidence

## REQUIRED INPUTS

You must derive this phase from the actual current repository, including at minimum:

- [routes/console.php](#)
- legacy import seeders
- continuity schema/runtime from PX05
- [migration\\_logs](#)
- [migration\\_rejections](#)
- [legacy\\_record\\_snapshots](#)
- media-related models/services
- page/SEO-related models/services
- current public route model
- any current missing-data export logic
- current imported content state relevant to continuity

You must build from work already done in the repo.

## REQUIRED IMPLEMENTATION OUTCOMES

By the end of this phase, the repository should have tooling that can produce at least:

Legacy URL inventory

- machine-readable list/export of relevant legacy public URL candidates
- enough metadata to support redirect planning and validation

#### Redirect validation

- a way to validate redirect continuity input before trusting it
- clear reporting of invalid/duplicate/conflicting rules

#### File continuity inventory

- machine-readable inventory/report of legacy file/document continuity state
- mapped vs unmapped visibility

#### Unresolved continuity reporting

- structured reporting of unresolved URL/file continuity issues
- actionable outputs for engineering and ops

#### Imported SEO/page continuity validation

- reports that identify imported pages/content not yet continuity-ready

#### Reconciliation reporting

- explicit reporting on ambiguous legacy areas that still need decisions or manual cleanup

#### VERIFICATION REQUIREMENTS

At the end of the phase, verify at minimum:

- a legacy URL inventory export can be generated successfully
- a redirect validation command/report can detect bad or conflicting input
- a file continuity inventory/report can be generated successfully
- unresolved continuity reporting can be generated successfully
- imported page SEO/continuity readiness reporting can be generated successfully
- outputs are machine-readable where appropriate
- new tooling aligns with existing command conventions and current repo architecture
- the reports are useful enough to support real migration planning, not just toy diagnostics

#### DONE WHEN

- Engineering can generate repeatable continuity inventories and validation reports without manual SQL-only workflows.
- Redirect map validation exists and can detect dangerous input before use.
- File/document continuity reporting exists and is actionable.
- Unresolved continuity is visible and queryable.
- Imported page SEO/continuity readiness can be assessed systematically.
- Ambiguous legacy structures have explicit reporting paths.

- The repository is ready for the final hardening, tests, launch validation, and rollback-prep phase.

## CONSTRAINTS

- Do not replace already-working import commands without cause.
- Do not build throwaway ad hoc scripts if reusable commands/services fit better.
- Do not mutate legacy source data.
- Do not hide unresolved continuity issues behind optimistic assumptions.
- Do not expand into final cutover execution in this phase.

## ARCHITECTURE RULES

- Service Layer is the only place for business logic.
- Console tooling should be consistent with existing repo command style.
- Models remain passive.
- Continuity and reporting logic should be centralized and testable.
- Use the current repository's import and continuity foundations as the base truth.

## IMPORTANT IMPLEMENTATION NOTE

This phase assumes:

- continuity schema/runtime from PX05 already exists
- homepage/page/admin foundations are already real
- import audit/export infrastructure already exists and should be reused

Do not behave as if migration tooling starts from zero.

You are extending an already-audited import foundation into a practical cutover-preparation toolkit.

## PX08-HARDENING-TESTS-LAUNCH — Final Hardening, Tests, Launch Validation, and Rollback

Preparation

DEPENDS ON

PX06-ADMIN-FILAMENT-COMPLETION

PX07-MIGRATION-BACKFILL

CONTEXT

Project: Syrian Private University website ([spu.edu.sy](https://spu.edu.sy))

Stack: Laravel 12, Filament v3, MySQL 8, Redis

Architecture rule: Service Layer is the only place for business logic

By the time this phase begins, the repository should already have:

- real public homepage and landing-page runtime
- real homepage CMS workflows
- real navigation/settings shell
- real SEO output
- continuity schema/runtime for redirects and file handling
- migration/backfill tooling for URL/file/reconciliation reporting
- a working admin/Filament foundation for the homepage/admin scope

This final phase is not for building new major architecture.

It is for converting the already-implemented foundation into something that is:

- testable
- cache-safe
- observable
- migration-rehearsal ready
- launch-check ready
- rollback-prepared

Current repo reality that must be treated as true:

- some tests already exist, but migration/continuity/SEO coverage is still the missing focus
- caching behavior already exists and must now be hardened, not invented from zero
- continuity logic now exists and must be validated
- launch readiness depends on verification and rollback preparation, not just code completeness
- storage/public delivery status must be verified as a real operational dependency

This phase is not for:

- full-site feature expansion
- additional module delivery outside current scope
- replacing already-working architecture without evidence

- final production cutover execution itself

## GOAL

Harden the current foundation so the project is ready for serious migration rehearsal and launch decision-making.

This includes:

- test coverage for launch-critical behavior
- cache hardening
- runtime validation tooling
- launch validation checklist/workflows
- rollback preparation
- production-readiness verification for the current homepage/admin foundation scope

## TASK

Complete the final hardening and readiness layer using the actual repository implementation built in prior phases.

You must do all of the following.

1. Review the current test and validation baseline

Inspect the existing repository test and validation setup before adding new work.

At minimum inspect:

- `tests/Feature/*`
- `tests/Unit/*`
- any test utilities/fakes/factories already present
- current cache-related behavior
- current preview behavior
- current SEO output behavior
- current continuity runtime and tooling
- current homepage/page/admin workflows
- any existing launch-oriented commands/checks already present

You must build from current repo reality.

2. Add launch-critical feature test coverage

Create or complete feature tests for the homepage/admin foundation and migration continuity scope.

At minimum cover:

1. locale middleware behavior

2. content-language and direction headers
3. admin auth redirect behavior
4. account lockout after 5 failed attempts
5. role restrictions for editor/super\_admin/faculty\_editor where relevant
6. homepage render from published data
7. homepage draft invisibility
8. scheduled homepage invisibility before publish time
9. homepage preview token behavior
10. landing-page render from published data
11. landing-page draft invisibility
12. landing-page schedule/publish behavior
13. breadcrumb generation
14. language-switch context preservation
15. canonical rendering
16. hreflang rendering
17. OG/meta rendering where applicable
18. sitemap output
19. robots.txt output
20. redirect continuity resolution
21. unresolved legacy request logging
22. file/document continuity resolution
23. menu depth enforcement
24. navigation/settings shell payload correctness
25. settings-driven cache invalidation
26. homepage publish cache invalidation
27. page publish SEO/page cache invalidation
28. media upload validation and rejection of unsafe files
29. audit logging for important write actions

Do not add superficial tests.

Focus on behavior that affects migration readiness and launch safety.

3. Add unit tests for core business rules

Add or complete unit tests where feature tests are not the best fit.

At minimum consider unit-level coverage for:

- slug uniqueness behavior if still relevant
- canonical URL generation
- hreflang generation/reciprocity behavior
- page visibility state rules
- homepage publish readiness validation
- menu depth validation
- redirect rule matching priority
- unresolved continuity classification logic
- settings payload resolution
- preview token expiry logic

Keep unit tests focused and meaningful.

#### 4. Harden cache behavior

Review and finalize the caching model for the current scope.

At minimum ensure:

- homepage cache behavior is deterministic
- landing-page cache behavior is deterministic
- preview bypass works reliably
- authenticated/admin bypass works reliably
- navigation/settings cache invalidation is targeted
- SEO-related output invalidation works where needed
- continuity behavior does not produce stale/incorrect public responses
- debug headers remain useful for validation if part of the current runtime design

If helpful, add or complete cache warm tooling for:

- homepage AR/EN
- top-level landing pages AR/EN
- shared navigation/settings payloads
- sitemap output if appropriate

#### 5. Verify storage/public delivery readiness

The project must be ready for public file/media continuity and normal media delivery.

At minimum:

- verify or enforce the expected storage/public delivery setup
- validate public URLs for representative media
- validate representative file/document continuity requests
- ensure this is reflected in test or validation workflows where possible

This must account for the actual runtime/storage configuration used by the app.

#### 6. Finalize audit coverage expectations

Ensure the important lifecycle actions are logged and testable.

At minimum ensure and verify audit coverage for actions such as:

- user login/logout/login failure/lock
- homepage update/draft/publish/schedule/unpublish
- page create/update/publish/schedule/unpublish
- menu create/update/reorder/toggle/delete/disable
- media upload/update/delete
- settings update

- user create/update/lock/unlock
- continuity-relevant admin actions where appropriate

Do not assume audit coverage exists; verify it.

## 7. Add launch validation tooling/checks

Create or complete practical launch-validation support for the current scope.

At minimum provide a repeatable way to validate:

- homepage public rendering
- landing-page public rendering
- AR/EN locale correctness
- canonical/hreflang correctness
- sitemap presence and correctness
- `robots.txt` correctness
- representative redirect continuity
- representative file continuity
- unresolved request observability
- admin preview safety
- cache behavior
- audit behavior

This may include:

- commands
- scripts
- structured test suites
- clearly documented validation workflow

The result must be operationally useful.

## 8. Add rollback preparation

This phase must prepare the repo/workflow for rollback-safe launch planning.

At minimum document or implement support for:

- rollback threshold definition
- cutover abort criteria
- pre-cutover snapshot expectation
- continuity rollback expectations
- how unresolved continuity spikes are monitored after cutover
- how cache and routing behavior can be safely reverted if needed

This does not mean executing rollback.

It means making the project operationally ready for rollback planning.

## 9. Produce a final launch-readiness checklist

Add or update a checklist that engineering can actually use before migration/cutover.

It should include at minimum:

- routing/runtime checks
- locale checks
- SEO checks
- continuity checks
- file/media checks
- admin checks
- cache checks
- audit checks
- staging noindex checks
- rollback readiness checks

This checklist must reflect actual current repo architecture, not the outdated original prompt pack assumptions.

## 10. Keep this phase limited to hardening and readiness

This phase is only for final hardening and readiness of the current homepage/admin foundation and continuity scope.

Do not expand into:

- new out-of-scope feature modules
- redesigning already-working runtime architecture without reason
- final live production cutover execution
- full archive implementation
- speculative analytics or marketing feature expansion

## REQUIRED INPUTS

You must derive this phase from the actual current repository, including at minimum:

- `tests/Feature/*`
- `tests/Unit/*`
- homepage/page/navigation/settings/media/admin services and controllers
- continuity runtime/services added in PX05
- backfill/reporting commands from PX07
- cache service and related middleware
- preview behavior
- storage/media behavior
- SEO rendering layer
- sitemap/robots behavior
- audit service/logging behavior
- current route/runtime behavior

You must build from actual prior-phase implementation, not generic assumptions.

## REQUIRED IMPLEMENTATION OUTCOMES

By the end of this phase, the repository should have these properties:

### Test coverage

- launch-critical homepage/admin/continuity behaviors are covered
- tests meaningfully validate migration-readiness risk areas

### Cache hardening

- public cache behavior is stable and predictable
- invalidation is targeted and verified
- preview/admin bypass behavior is reliable

### Operational validation

- there is a repeatable way to validate public runtime, SEO, continuity, and admin safety
- representative continuity cases can be checked quickly

### Rollback preparation

- rollback expectations are explicit
- launch decision-making has defined validation and abort criteria

### Launch readiness

- the project has a real engineering checklist for migration rehearsal and go/no-go review

## VERIFICATION REQUIREMENTS

At the end of the phase, verify at minimum:

- test suite covers the critical behaviors listed above
- homepage/page/public visibility behavior is tested
- preview behavior is tested
- canonical/hreflang behavior is tested
- sitemap and [robots.txt](#) are tested
- redirect continuity is tested
- unresolved continuity logging is tested
- file/document continuity is tested
- menu/settings/media constraints are tested
- cache invalidation behavior is tested
- audit log behavior is tested
- launch-readiness checklist exists and is aligned to the actual repo
- rollback preparation exists and is explicit

## DONE WHEN

- The homepage/admin foundation is hardened enough for serious migration rehearsal.
- Launch-critical behaviors have meaningful test coverage.
- Cache and continuity behavior are validated and predictable.
- Launch validation is operationally usable.
- Rollback preparation is explicit enough to support go/no-go planning.
- The rebuilt phase plan is complete end-to-end for the current scope.

## CONSTRAINTS

- Do not expand into out-of-scope feature delivery.
- Do not replace already-correct architecture without evidence.
- Do not add shallow tests that do not validate real migration risk.
- Do not treat storage/continuity/caching as assumptions; verify them.
- Do not perform actual cutover in this phase.

## ARCHITECTURE RULES

- Service Layer is the only place for business logic.
- Controllers remain thin.
- Models remain passive.
- Tests should validate behavior, not implementation details unnecessarily.
- Operational readiness should be grounded in actual runtime architecture and continuity behavior.
- Use current repository implementation as the source of truth.

## IMPORTANT IMPLEMENTATION NOTE

This phase assumes:

- the real runtime exists
- homepage CMS exists
- shared shell exists
- SEO and continuity exist
- admin/Filament completion exists
- migration backfill/reporting tooling exists

Do not behave as if this phase starts from zero.

You are hardening and validating an already-implemented foundation so it can support migration rehearsal and launch decision-making.